NT

NovaTicket

Memoria Técnica

Autor: Justin Esaac

Abril 2026

Gestión y Venta de Entradas para Eventos · Java 21 · Spring Boot · MySQL

Índice de Contenidos

1. **Descripción del Proyecto** 3
2. **Tecnologías Utilizadas** 3
3. **Explicación de la Arquitectura** 3
4. **Instrucciones de Despliegue** 4
5. **Diagramas** 5
   - 5.1 Diagrama de Clases 5
   - 5.2 Modelo Entidad-Relación 6
   - 5.3 Diagrama de Secuencia – Usuario 7
   - 5.4 Diagrama de Secuencia – Admin 8

Memoria Tecnica - NovaTicket

1. Descripcion del proyecto

NovaTicket es una aplicacion web orientada a la gestion y venta de entradas para eventos de distintos tipos, como conciertos, obras de teatro y museos. El sistema contempla dos perfiles principales: cliente y administrador. El cliente puede consultar eventos disponibles, filtrar por categoria y disponibilidad, acceder al detalle de cada evento, anadir entradas al carrito, completar la compra y revisar su historial. El administrador dispone de un panel de control para gestionar de forma centralizada lugares, asientos, eventos, tickets, usuarios y compras.

El objetivo del proyecto es ofrecer una experiencia de compra clara y funcional, con control de stock de entradas, asignacion de asientos y trazabilidad de operaciones mediante historial de compras. Ademas, se ha trabajado el aspecto visual del frontend para mantener coherencia de diseno, accesibilidad basica y comportamiento responsive en distintos tamanos de pantalla.

2. Tecnologias utilizadas

- **Java 21** como lenguaje principal del backend.
- **Spring Boot** para arranque de aplicacion y estructura web MVC.
- **Spring Web** para controladores, rutas y renderizado de vistas.
- **Thymeleaf** para plantillas HTML del lado servidor.
- **Spring JDBC** para acceso a datos con capa DAO personalizada.
- **MySQL** como base de datos relacional.
- **Bootstrap 5** y **Bootstrap Icons** para estilos e interfaz.
- **JavaScript** para interacciones de frontend (filtros, calculos y UI).
- **Maven** para gestion de dependencias y ciclo de build.

3. Explicacion de la arquitectura

La aplicacion sigue una arquitectura por capas basada en el patron MVC. En la capa de presentacion, los *controllers* gestionan peticiones HTTP y preparan la informacion para las vistas Thymeleaf. La capa de negocio se ubica en *services*, donde se concentra la logica funcional de compra, validaciones y gestion de recursos. El acceso a datos se realiza mediante una capa *repository/DAO* con implementaciones JDBC que encapsulan las operaciones CRUD sobre MySQL. La capa *model* define las entidades del dominio (usuario, evento, ticket, compra, asiento, lugar, etc.) y sus relaciones.

A nivel de seguridad y flujo, se usa sesion HTTP para identificar al usuario autenticado y un interceptor para proteger rutas administrativas y acciones de compra. En frontend, la plantilla principal integra estilos globales, componentes responsivos y scripts para filtros y validaciones visuales sin recargar la pagina innecesariamente.

4. Instrucciones de despliegue

1. Crear la base de datos **novaticket** en MySQL.
2. Ejecutar los scripts SQL de inicializacion en **DOCS/ini_db.sql** y, opcionalmente, datos de prueba en **DOCS/data.sql**.
3. Revisar credenciales de conexion en **src/main/resources/application.properties**.
4. Compilar y arrancar la aplicacion desde la raiz del proyecto con Maven Wrapper:

```
./mvnw spring-boot:run
```

En Windows puede usarse: **mvnw.cmd spring-boot:run**. Una vez iniciado, acceder a la aplicacion desde el navegador en la URL configurada por Spring Boot (por defecto, puerto 8080).

5. Diagramas

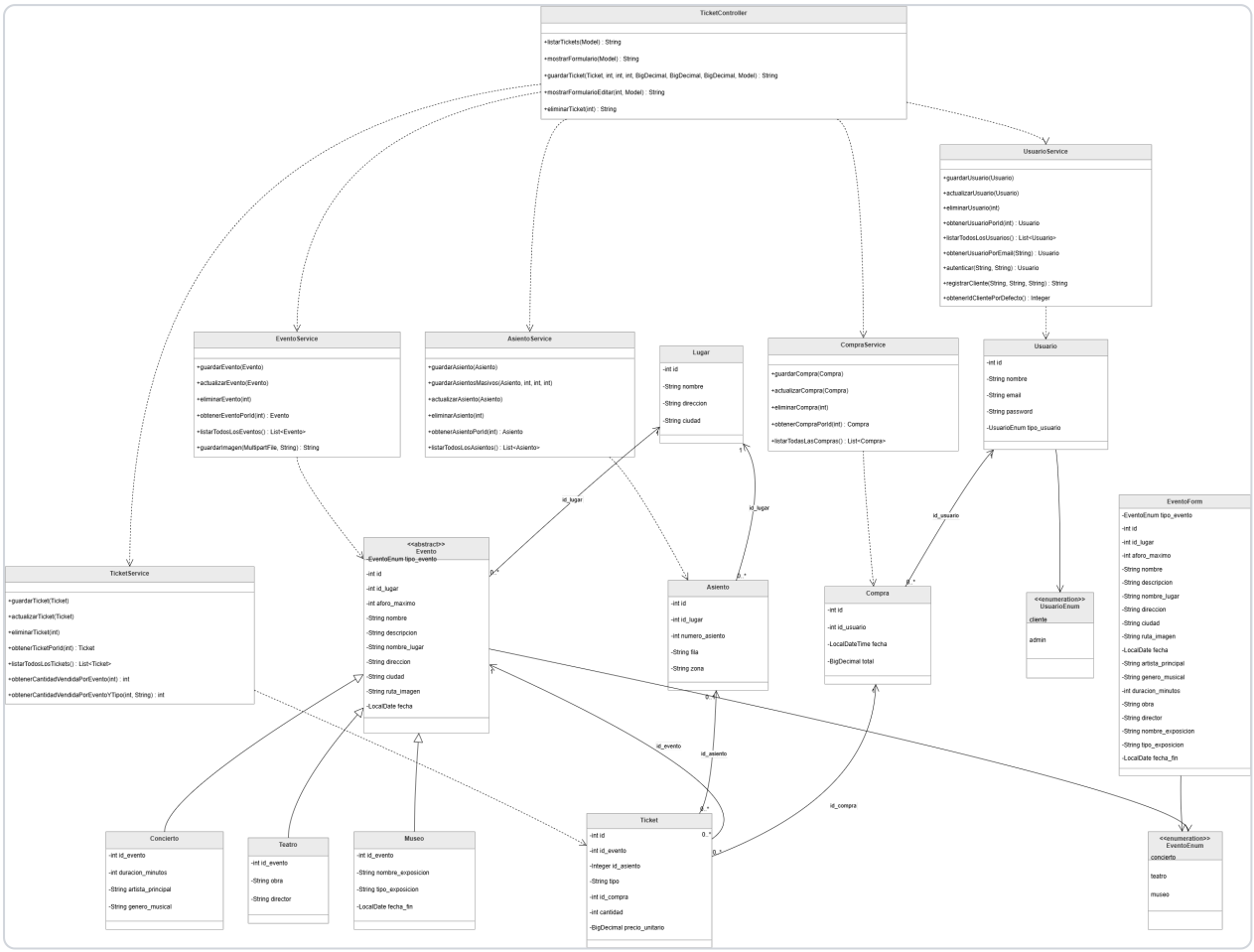

Figura 1. Diagrama de clases (archivo: DIAGRAMA DE CLASES.png)

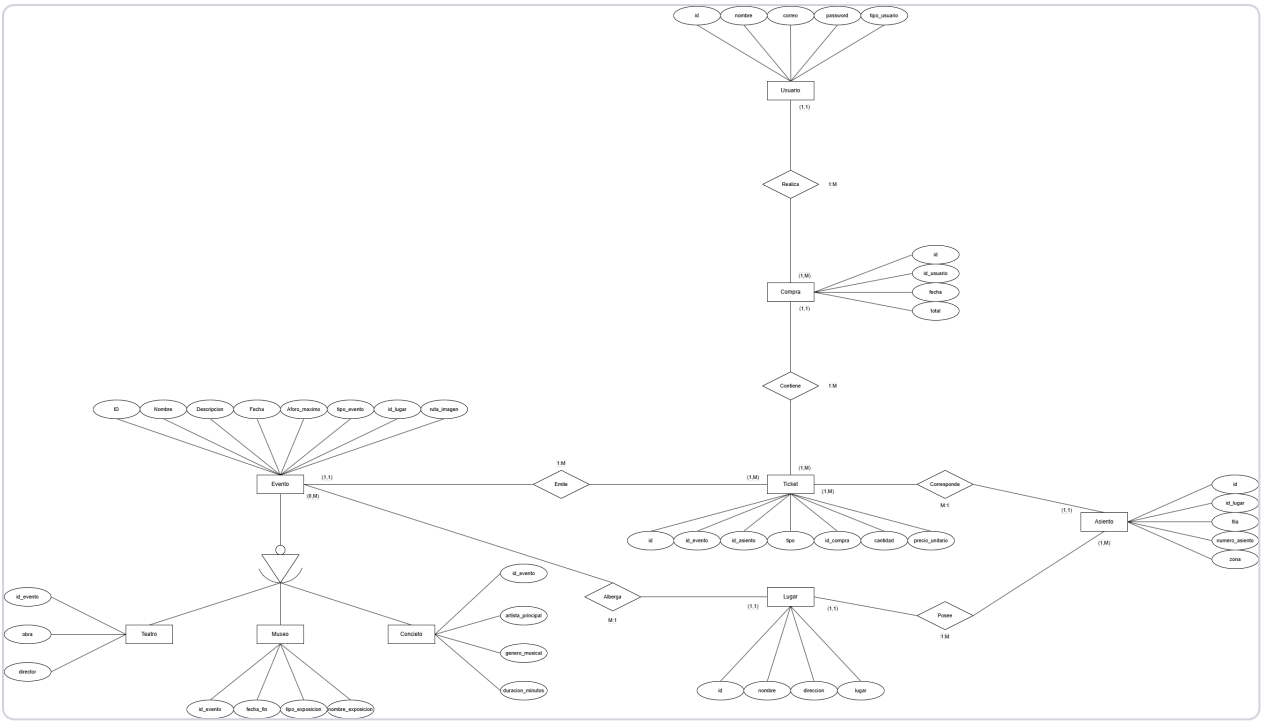

Figura 2. Modelo Entidad-Relacion (archivo: MODELO ENTIDAD-RELACION.png)

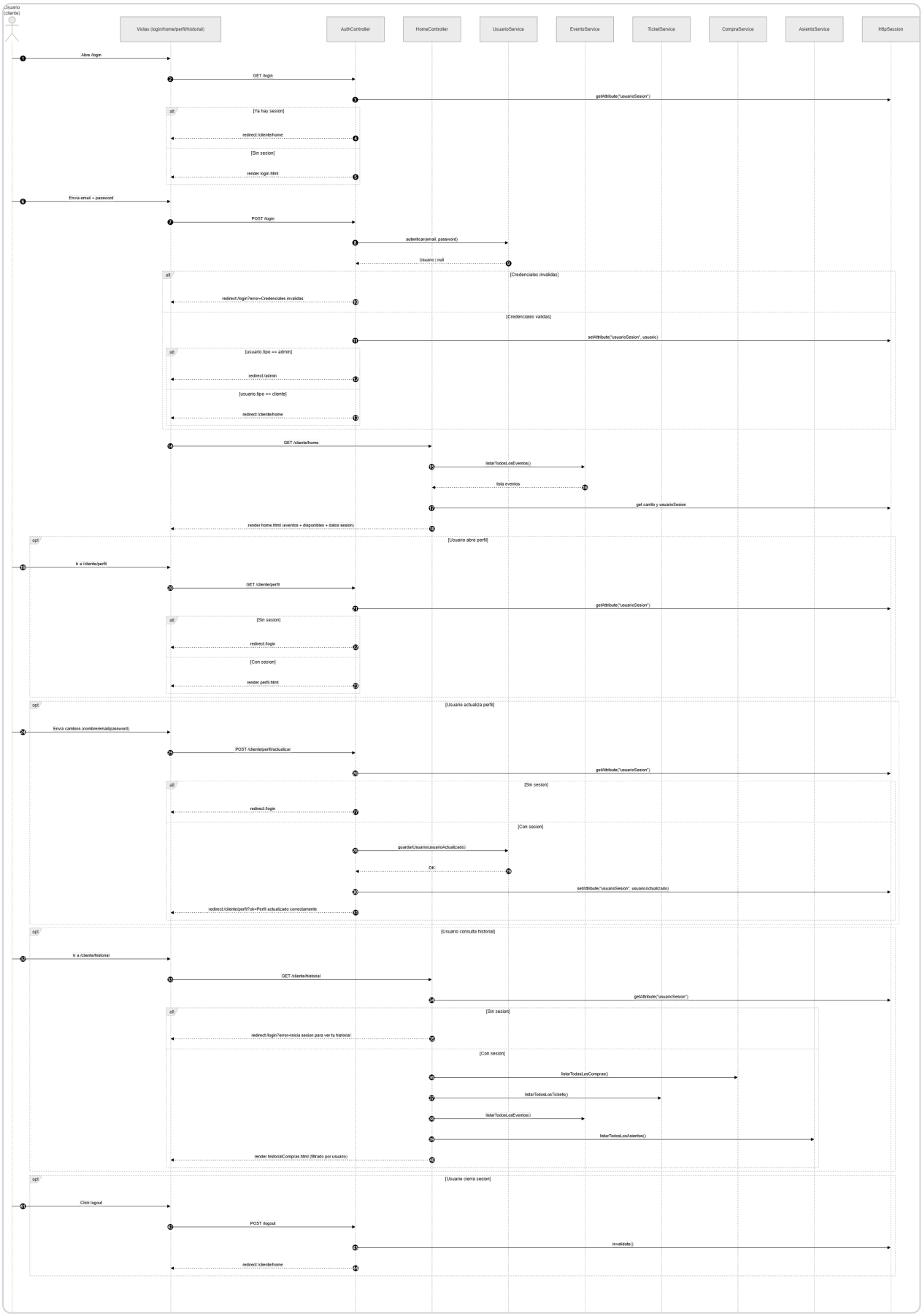

Figura 3. Diagrama de secuencia usuario (opcional) - archivo: DIAGRAMA DE SECUENCIA USUARIO.drawio.png

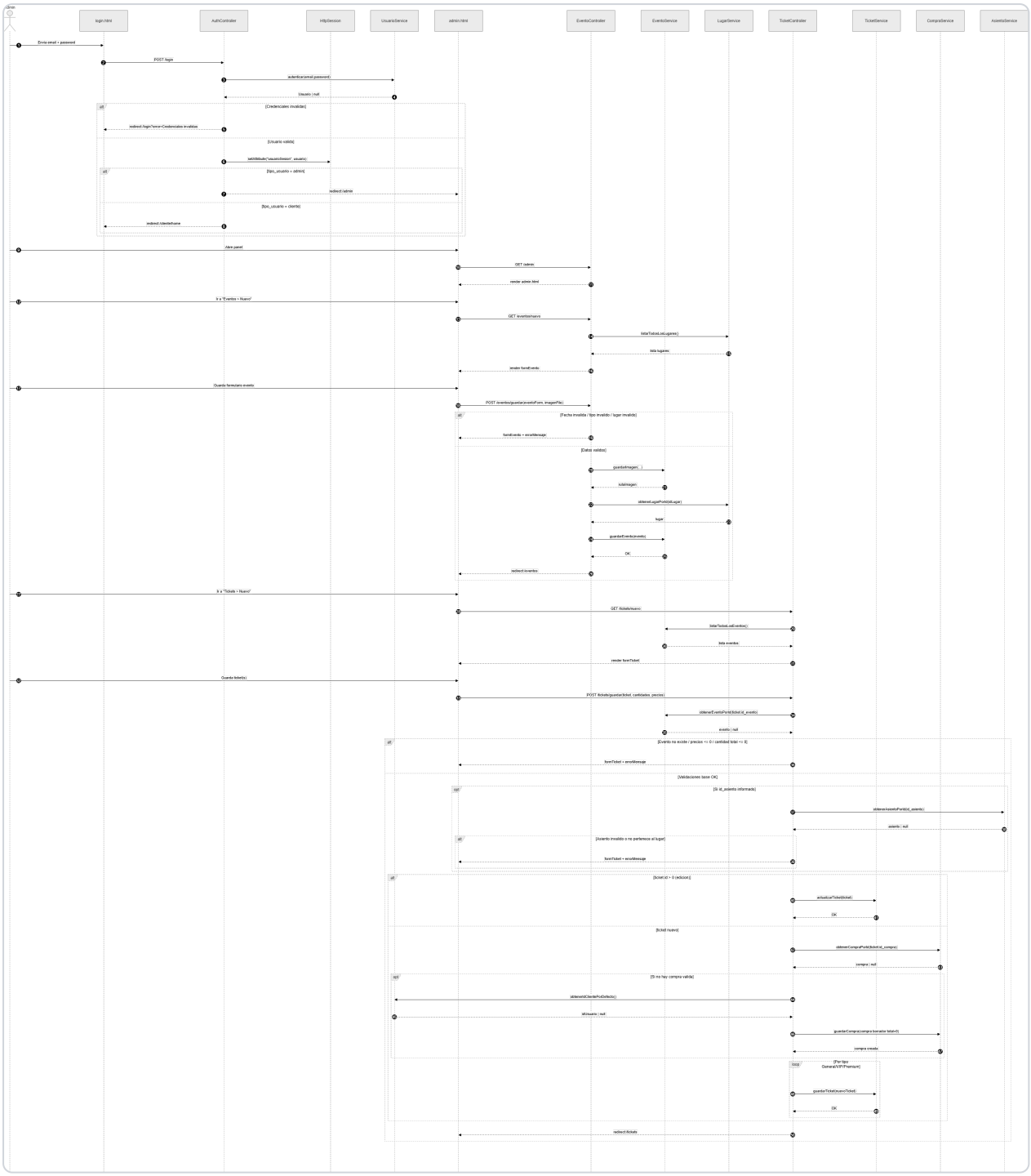

Figura 4. Diagrama de secuencia admin (opcional) - archivo: DIAGRAMA DE SECUENCIA ADMIN.drawio.png